

ALPS

AI Learning Path Selector

*Sistema de Optimización de Rutas de Aprendizaje
con Integración de LLM*

Proyecto Final de Inteligencia Artificial 2025–2026

Tema 9: Construcción de Rutas de Aprendizaje

Lianny de la Caridad Revé Valdivieso

Facultad de Matemática y Computación
Universidad de La Habana
Curso 2025–2026

13 de junio de 2026

Índice

1. Descripción del Problema	2
1.1. Marco de simulación	2
1.2. Entradas y salidas	2
2. Modelado Formal	2
2.1. Variables de decisión	3
2.2. Restricciones duras	3
2.3. Función objetivo	3
2.4. Subproblema de ordenamiento	3
3. Dataset	3
3.1. Generación del dataset sintético	3
3.2. Estructura y propiedades	3
4. Diseño del Sistema	4
4.1. Cierre de dependencias y postprocesamiento	4
4.2. Algoritmo 1: Greedy (baseline)	4
4.3. Algoritmo 2: Hill Climbing con reinicios aleatorios	5
4.4. Óptimo exacto como referencia	5
4.5. Ordenamiento topológico de la selección	5
5. Rol del LLM en el Sistema	6
5.1. Diseño del prompt	6
5.2. Integración arquitectónica	7
5.3. Limitaciones del scoring	7
6. Metodología Experimental	8
6.1. Objetivos de aprendizaje	8
6.2. Presupuestos de tiempo	8
6.3. Métricas de comparación	8
6.4. Análisis Monte Carlo del Hill Climbing	8
7. Resultados y Análisis	9
7.1. Scores del LLM y efecto del dependency boost	9
7.2. Experimento 1: Objetivo ML Engineer	10
7.3. Experimento 2: Objetivo NLP Researcher	11
7.4. Análisis Monte Carlo del Hill Climbing	12
8. Limitaciones y Posibles Mejoras	14
8.1. Dependency drag	14
8.2. Compresión de scores en modelos pequeños	14
8.3. Exploración limitada del Hill Climbing	15
8.4. Dataset sintético de tamaño reducido	15
9. Conclusiones	15

1 Descripción del Problema

El aprendizaje de temáticas complejas como la Inteligencia Artificial implica un conjunto amplio de recursos educativos con relaciones de dependencia: no es posible comprender redes neuronales sin álgebra lineal y optimización, ni usar NumPy sin Python. A esto se suma que los estudiantes disponen de presupuestos de tiempo limitados y objetivos de aprendizaje heterogéneos.

ALPS (AI Learning Path Selector) resuelve el problema de construir automáticamente una ruta de aprendizaje personalizada: dado un catálogo de recursos educativos con sus dependencias, un objetivo en lenguaje natural y un presupuesto de tiempo en horas, el sistema selecciona el subconjunto óptimo de recursos y lo ordena en una secuencia válida que maximiza la utilidad para el usuario sin exceder su presupuesto.

1.1 Marco de simulación

Siguiendo el marco de la simulación computacional, ALPS puede describirse como el modelo computacional de un sistema de aprendizaje formal:

- **Entidades:** recursos educativos, caracterizados por nombre, descripción, duración y prerrequisitos.
- **Relaciones estructurales:** dependencias entre recursos (grafo dirigido acíclico).
- **Relaciones funcionales:** utilidad de cada recurso para un objetivo dado (evaluada por el LLM).
- **Proceso:** construcción de una secuencia óptima de estudio que satisface las restricciones.

El sistema es *abierto* (recibe entradas: objetivo y presupuesto; emite salidas: ruta de aprendizaje) y *no-determinista* en su componente Hill Climbing, cuyo comportamiento depende de variables aleatorias uniformes generadas en cada ejecución.

1.2 Entradas y salidas

Entradas

- (1) objetivo de aprendizaje en lenguaje natural, (2) presupuesto de tiempo en horas, (3) catálogo de recursos con metadatos y dependencias.

Salida

secuencia ordenada de recursos que respeta todas las restricciones y maximiza la utilidad total para el objetivo dado.

2 Modelado Formal

El problema se formula como optimización combinatoria con restricciones. Sea $R = \{r_1, \dots, r_n\}$ el conjunto de recursos, d_i la duración en horas de r_i , y u_i su puntuación de utilidad para el objetivo del usuario.

2.1 Variables de decisión

Para cada recurso r_i se define una variable binaria:

$$x_i \in \{0, 1\}, \quad x_i = \begin{cases} 1 & \text{si } r_i \text{ se incluye en la ruta} \\ 0 & \text{en caso contrario} \end{cases}$$

2.2 Restricciones duras

Cierre de dependencias. Si un recurso se incluye, todos sus prerequisites transitivos deben incluirse también:

$$\forall r_i : \quad x_i = 1 \wedge r_j \in \text{prereqs}_{\text{trans}}(r_i) \implies x_j = 1$$

Presupuesto. La suma de duraciones no puede superar el presupuesto T :

$$\sum_{i=1}^n x_i d_i \leq T$$

2.3 Función objetivo

Se maximiza la suma de utilidades de los recursos seleccionados:

$$\text{máx} \sum_{i=1}^n x_i u_i$$

2.4 Subproblema de ordenamiento

Una vez determinado el subconjunto S , la secuencia de estudio debe respetar que ningún recurso aparezca antes que sus prerequisites. Se calcula un *ordenamiento topológico* del subgrafo inducido por S mediante el algoritmo de Kahn (BFS sobre grados de entrada) en tiempo $O(|S| + |E|)$.

3 Dataset

3.1 Generación del dataset sintético

Se generó un dataset sintético de **21 recursos educativos** en el dominio de Inteligencia Artificial y Machine Learning. La elección de un dataset sintético permite controlar con precisión la estructura del grafo de dependencias y las propiedades experimentales deseadas.

El script `generate_dataset.py` genera y valida automáticamente el dataset verificando: (1) ausencia de IDs duplicados, (2) integridad referencial de todas las dependencias, y (3) ausencia de ciclos mediante DFS con pila activa. La ausencia de ciclos es condición necesaria para que el ordenamiento topológico sea posible.

3.2 Estructura y propiedades

El dataset se organiza en cuatro niveles de profundidad que reflejan la jerarquía natural del conocimiento en IA/ML:

- **Nivel 0** (5 recursos, sin prerequisites): Python básico, Álgebra lineal, Cálculo diferencial, Probabilidad y estadística, Lógica y matemática discreta.
- **Nivel 1** (5 recursos): NumPy y Pandas, Visualización de datos, Machine learning clásico, Optimización matemática, Preprocesamiento de texto.
- **Nivel 2** (5 recursos): Redes neuronales densas, Evaluación de modelos, Word embeddings, CNN, RNN/LSTM.
- **Nivel 3** (6 recursos): Arquitectura Transformer, Fine-tuning de LLMs, NLP clásico, Aprendizaje por refuerzo, MLOps y despliegue, Interpretabilidad.

Propiedades que justifican la idoneidad del dataset para el problema:

- 21 recursos con duraciones heterogéneas entre 2h y 7h (total: 90h), lo que hace genuinamente restrictiva la restricción de presupuesto para presupuestos típicos de 15–30h.
- 13 de los 21 recursos tienen 2 o más prerequisites directos, creando dependencias múltiples que generan tensiones reales en la selección.
- Las descripciones están redactadas en lenguaje natural específico, permitiendo al LLM evaluar utilidad diferencial según distintos objetivos de aprendizaje.
- La estructura del grafo permite explorar el fenómeno de *dependency drag*: recursos de alta utilidad al final de cadenas largas de dependencias son inaccesibles con presupuestos pequeños.

4 Diseño del Sistema

4.1 Cierre de dependencias y postprocesamiento

Cierre transitivo. La función `compute_closure` implementa un DFS iterativo que, dado un recurso objetivo, devuelve el conjunto completo de recursos que deben incluirse para poder seleccionarlo (el recurso más todos sus ancestros transitivos).

Dependency boost. El LLM tiende a subvalorar recursos fundacionales porque evalúa relevancia directa al objetivo, ignorando su valor como prerequisite. Para corregir este sesgo se aplica un *boost* post-proceso:

$$u_{\text{boost}}(r_i) = \min\left(10, 0, u_{\text{raw}}(r_i) + 0,3 \cdot \max\{u(r_j) : r_i \in \text{closure}(r_j)\}\right)$$

Cada recurso recibe un boost proporcional a la utilidad máxima de todos los recursos que transitivamente dependen de él. El factor $\alpha = 0,3$ se determinó empíricamente.

Sin umbral de utilidad. El sistema no impone ningún piso mínimo de utilidad: la selección se rige únicamente por la función objetivo. Una versión preliminar descartaba como objetivos los recursos por debajo de un umbral ($\text{boosted} < 4,0$) para evitar “relleno” de baja utilidad, pero al validar contra el óptimo exacto se comprobó que ese umbral dejaba utilidad alcanzable sin explotar (sección 7); el propio objetivo de maximización ya penaliza los recursos de baja utilidad, de modo que el umbral era innecesario y subóptimo. Se eliminó.

4.2 Algoritmo 1: Greedy (baseline)

En cada iteración calcula la eficiencia marginal de cada candidato:

$$\text{eficiencia}(r_i) = \frac{\text{utilidad nueva}}{\text{horas nuevas}}$$

Selecciona el candidato de mayor eficiencia que quepa en el presupuesto, expande su cierre de dependencias y repite hasta que ningún candidato quepa. Es determinista y rápido pero sus decisiones son irrevocables.

4.3 Algoritmo 2: Hill Climbing con reinicios aleatorios

Opera sobre el espacio de soluciones completas mediante tres tipos de movimiento:

- **ADD:** incluir un recurso no seleccionado junto con su cierre, si el costo cabe en el presupuesto.
- **REMOVE:** eliminar un recurso hoja (sin dependientes en la selección actual).
- **SWAP:** eliminar una hoja y agregar un recurso diferente en un único paso, liberando horas para financiar el nuevo recurso. Este es el movimiento crítico para escapar óptimos locales.

En cada iteración se mueve al vecino con mayor utilidad si supera a la solución actual; si no, detiene la búsqueda local. El proceso se repite desde 5 puntos de inicio aleatorios independientes, conservando la mejor solución global.

Conexión con metaheurísticas: los reinicios aleatorios implementan *exploración* para escapar óptimos locales; la búsqueda en el vecindario implementa *explotación* de una región prometedora.

4.4 Óptimo exacto como referencia

Comparar dos heurísticas entre sí solo dice cuál es mejor, no cuán lejos están del máximo alcanzable. Para medir la *brecha de optimalidad* (*optimality gap*) de Greedy y Hill Climbing se calcula el óptimo exacto del problema, es decir, del *knapsack con restricciones de precedencia*:

$$\max \sum_i u_i x_i \quad \text{s.a.} \quad \sum_i d_i x_i \leq T, \quad x_i \leq x_j \quad \forall r_j \in \text{prereqs}(r_i).$$

Se resuelve por **branch and bound** recorriendo los recursos en orden topológico (prerequisitos antes que dependientes). Un recurso solo puede incluirse cuando todos sus prerequisitos ya están incluidos —garantizando que todo estado visitado es una selección cerrada y factible— y una cota optimista (suma de las utilidades positivas restantes) poda las ramas que no pueden superar la mejor solución hallada. El procedimiento es *dependency-free*, exacto y muy rápido a esta escala (~ 3 ms por presupuesto); la misma formulación se traslada directamente a un solver ILP para escalar a instancias de 50+ recursos.

El óptimo exacto como herramienta de validación. Disponer del máximo demostrable no solo permite cuantificar la brecha de las heurísticas: también sirvió para auditar el propio diseño del sistema. Fue al comparar contra él que se detectó que el umbral de utilidad (sección 4.1) producía soluciones subóptimas, lo que motivó su eliminación.

4.5 Ordenamiento topológico de la selección

Una vez que cualquiera de los dos algoritmos determina el subconjunto óptimo S , resolver el problema de selección no es suficiente: la secuencia de estudio debe garantizar que ningún recurso aparezca antes que sus prerequisitos. Este constituye el **segundo subproblema** del sistema, independiente del algoritmo de selección usado.

Se implementa el **algoritmo de Kahn**, basado en BFS sobre los grados de entrada (*in-degree*) de los nodos del subgrafo inducido por S :

1. Calcular el in-degree de cada $r_i \in S$, contando únicamente las aristas entre nodos de S (los prerrequisitos externos a S no aplican).
2. Inicializar una cola Q con todos los $r_i \in S$ cuyo in-degree es 0 —recursos que no dependen de ningún otro recurso en la selección actual.
3. Mientras Q no esté vacía:
 - a. Extraer r_i de Q y añadirlo al ordenamiento final.
 - b. Para cada $r_j \in S$ tal que $r_i \in \text{prereqs}(r_j)$, decrementar $\text{in-degree}(r_j)$; si llega a 0, añadir r_j a Q .

El algoritmo opera en tiempo $O(|S| + |E_S|)$, donde $|E_S|$ es el número de aristas del subgrafo inducido. La ausencia de ciclos en el dataset —validada en la generación mediante DFS con pila activa— garantiza que el ordenamiento siempre es completo: $|\text{ordenamiento}| = |S|$.

El ordenamiento topológico se aplica de forma idéntica a la salida de *ambos* algoritmos. La validez de la secuencia final es independiente de la estrategia de selección usada.

5 Rol del LLM en el Sistema

El LLM (llama-3.1-8b-instant vía Groq API) cumple el rol de puente entre el texto en lenguaje natural y los valores numéricos que requiere el algoritmo de optimización. Para cada recurso recibe el objetivo del usuario, el nombre y descripción del recurso, y su duración; y produce un score decimal entre 0.0 y 10.0 que pasa a ser el coeficiente u_i en la función objetivo.

5.1 Diseño del prompt

El prompt enviado al modelo para cada recurso es el siguiente:

```
You are an expert in AI/ML curriculum design
and personalized learning.

User's learning goal:
"{goal}"

Resource to evaluate:
- Name: {resource_name}
- Description: {resource_description}
- Duration: {duration_hours} hours

Rate how useful this resource is for reaching
the user's goal. Consider its direct relevance
to the goal -- resources that do not directly
address the goal should score lower, even if
they are generally useful.

Reply with ONLY a decimal number between 0.0
and 10.0. No other text.
Valid response examples: 8.5 | 3.0 | 9.5 | 2.0
```

Cada decisión de diseño responde a una restricción concreta:

- **Prompt corto y sin rúbrica:** durante la experimentación se comprobó que prompts con rúbricas detalladas (rangos de score con descripción de cada nivel) producen compresión de scores en modelos de tamaño reducido: el modelo asigna valores similares a todos los recursos (≈ 8.2), eliminando la diferenciación necesaria. Un prompt directo y minimalista produce mejor separación.
- **Instrucción de formato estricta:** se solicita explícitamente un único número decimal sin texto adicional, junto con ejemplos de respuesta válida. Esto reduce la tasa de respuestas malformadas que requieren parsing de recuperación.
- **Temperatura 0.1:** valor bajo que produce salidas más deterministas y consistentes entre llamadas sucesivas para el mismo recurso y objetivo, reduciendo la varianza del scoring.
- **Evaluación de relevancia directa:** la instrucción prioriza la relevancia directa al objetivo. La corrección por valor fundacional se delega al *dependency boost* algorítmico (sección 4), separando responsabilidades entre el LLM y el sistema.

5.2 Integración arquitectónica

Los 21 recursos se scorean secuencialmente con una pausa de 0.5s entre llamadas para respetar los límites de tasa de la API gratuita de Groq. Los scores se persisten en disco como caché por objetivo (un archivo `scores_<hash>.json` por cada objetivo distinto), evitando recálculos en corridas sucesivas: cada objetivo se evalúa con el LLM una sola vez.

Decisión metodológica: fijación de los cachés. El scoring del LLM *no es reproducible entre corridas*, ni siquiera con temperatura 0.1: re-evaluar el mismo objetivo produce scores crudos que varían en ± 0.5 –1.0 puntos, y esa variación se propaga por el *dependency boost* hasta las utilidades y el óptimo. Como toda la cadena numérica del informe depende de unos scores concretos, se fija una única corrida por objetivo: los dos cachés (`scores_<hash>.json` para ML Engineer y NLP Researcher) se versionan en el repositorio y *todas* las tablas de este documento se regeneran a partir de ellos. Así el informe es reproducible al 100 % desde los artefactos commiteados, sin depender de la API ni de la variabilidad del modelo.

5.3 Limitaciones del scoring

- **Subvaloración de recursos fundacionales:** el modelo evalúa relevancia directa ignorando el valor como prerequisite. Corregido mediante el *dependency boost*.
- **Compresión de scores:** con prompts complejos, el modelo asigna scores similares a todos los recursos, eliminando la diferenciación necesaria. Mitigado mediante prompt minimalista y temperatura baja (0.1).
- **No-reproducibilidad entre corridas:** a igual objetivo y temperatura, los scores crudos varían entre llamadas. Mitigado fijando y versionando los cachés (ver nota anterior), de modo que los resultados reportados sean estables y auditables.

6 Metodología Experimental

El diseño experimental compara el comportamiento de ambos algoritmos bajo dos objetivos de aprendizaje cualitativamente distintos y tres niveles de presupuesto, para un total de **12 configuraciones** (2 objetivos $\times$ 3 presupuestos $\times$ 2 algoritmos). En cada una de las 6 combinaciones objetivo-presupuesto se calcula además el óptimo exacto como referencia para medir la brecha de optimalidad.

6.1 Objetivos de aprendizaje

Objetivo 1 (ML Engineer)

“I want to work as an ML engineer building and deploying models in production”. Prioriza herramientas prácticas (MLOps, redes neuronales, NumPy) con recursos de alta utilidad distribuidos en distintos niveles del grafo.

Objetivo 2 (NLP Researcher)

“I want to research NLP and understand how large language models work internally”. Concentra la alta utilidad en recursos muy avanzados (Transformers, RNN, Word Embeddings) que requieren cadenas de dependencias de 40–52 horas.

6.2 Presupuestos de tiempo

Se evalúan tres presupuestos: **15h** (restrictivo), **20h** (moderado) y **30h** (amplio). La selección cubre el rango en que la restricción de tiempo es significativa (el total del dataset es 90h).

6.3 Métricas de comparación

- **Utilidad total:** suma de scores boosted de los recursos seleccionados.
- **Horas utilizadas:** total de horas de la ruta resultante vs. el presupuesto disponible.
- **Recursos seleccionados:** cardinalidad del conjunto seleccionado.
- **Iteraciones (HC):** número total de pasos de búsqueda realizados.
- **Brecha de optimalidad:** porcentaje de la utilidad óptima (calculada exactamente por branch and bound) que alcanza cada heurística. Una heurística con 100 % es demostrablemente óptima para esa instancia.

6.4 Análisis Monte Carlo del Hill Climbing

El Hill Climbing es un algoritmo no-determinista: su resultado depende de las permutaciones aleatorias uniformes utilizadas para construir las soluciones iniciales de cada reinicio. Una única ejecución es simplemente una muestra del proceso estocástico subyacente, no una estimación fiable del rendimiento real.

Para caracterizar estadísticamente el comportamiento del HC se aplica un análisis de Monte Carlo: se ejecutan $N = 30$ réplicas independientes con semillas distintas (semilla i para la réplica i , $i \in \{0, \dots, 29\}$), garantizando reproducibilidad e independencia estadística.

Cada réplica produce una observación de la variable aleatoria U = utilidad de la mejor solución encontrada. Las 30 observaciones permiten estimar:

- **Media muestral:** estimador de $\mathbb{E}[U]$, la utilidad esperada del HC.

- **Desviación estándar muestral:** variabilidad del algoritmo entre ejecuciones.
- **Rango** [mín, máx]: valores extremos observados en las 30 réplicas.
- **Intervalo de confianza del 95 %:** usando la distribución t de Student con $N - 1 = 29$ grados de libertad.

La fórmula del intervalo de confianza es:

$$\text{IC}_{95\%} = \left[\bar{x} - t \cdot \frac{s}{\sqrt{N}}, \bar{x} + t \cdot \frac{s}{\sqrt{N}} \right], \quad t = t_{0,025,29} = 2,045$$

donde s es la desviación estándar muestral.

Conexión con simulación: las 30 réplicas del HC constituyen una simulación de Monte Carlo del proceso estocástico de optimización. La variable aleatoria U se genera a partir de permutaciones uniformes aleatorias de los recursos —generación de variables aleatorias discretas uniformes— y su distribución se analiza con las técnicas estándar de análisis estadístico de simulaciones.

7 Resultados y Análisis

7.1 Scores del LLM y efecto del dependency boost

La siguiente tabla muestra una selección representativa de los scores del LLM (raw) y tras aplicar el dependency boost, para ambos objetivos. Se evidencia la diferenciación entre objetivos y el efecto corrector del boost sobre recursos fundacionales.

Cuadro 1: Scores raw vs. boosted por objetivo (selección representativa).

Recurso	ML Engineer			NLP Researcher		
	Raw	Boost	Δ	Raw	Boost	Δ
Python básico	2.5	5.0	+2.5	0.5	3.4	+2.9
Álgebra lineal	6.5	9.1	+2.6	2.5	5.3	+2.8
Cálculo diferencial	6.5	9.1	+2.6	2.5	5.3	+2.8
Probabilidad y estad.	2.5	5.0	+2.5	1.0	3.9	+2.9
NumPy y Pandas	7.5	10.0	+2.5	0.5	3.4	+2.9
Machine learning clásico	4.5	7.0	+2.5	0.5	3.4	+2.9
Optimización matemática	6.5	9.1	+2.6	0.5	3.4	+2.9
Preprocesamiento de texto	4.5	7.0	+2.5	4.5	7.3	+2.8
Redes neuronales densas	8.5	10.0	+1.5	2.5	5.3	+2.8
Word embeddings	6.5	9.1	+2.6	8.5	10.0	+1.5
Redes recurrentes (RNN)	7.5	10.0	+2.5	8.5	10.0	+1.5
Arquitectura Transformer	6.5	9.1	+2.6	9.5	10.0	+0.5
Fine-tuning de LLMs	8.5	8.5	—	8.5	8.5	—
MLOps y despliegue	8.5	8.5	—	0.0	0.0	—
CNN	7.5	7.5	—	0.5	0.5	—

El dependency boost corrige efectivamente la subvaloración de recursos fundacionales. *Cálculo diferencial* recibe raw 2.5 para NLP Researcher —el LLM no lo ve directamente relevante— pero

al ser requisito transitivo de *Transformers* (raw 9.5), su score boosted sube a 5.3. El mismo mecanismo eleva a *NumPy* y *Pandas* de 0.5 a 3.4 para NLP, reconociendo su papel como base de prácticamente toda la pila de ML.

La diferenciación por perfil es nítida: *MLOps* obtiene 8.5 para ML Engineer pero 0.0 para investigación NLP, y de forma simétrica *Transformers*, *RNN* y *Word Embeddings* (raw 8.5–9.5 para NLP) bajan a 6.5–7.5 para ML Engineer. Esto demuestra que el LLM evalúa la utilidad según el objetivo y no de forma absoluta.

7.2 Experimento 1: Objetivo ML Engineer

Cuadro 2: Comparación Greedy vs. Hill Climbing vs. óptimo exacto — objetivo ML Engineer.

Presup.	Greedy			Hill Climbing				Óptimo
	Horas	Util.	% ópt	Horas	Util.	Iter.	% ópt	Util.
15h	14h	33.65	100 %	15h	33.65	8	100 %	33.65
20h	19h	42.70	100 %	20h	42.70	10	100 %	42.70
30h	30h	59.30	96.7 %	30h	61.35	12	100 %	61.35

Con la función objetivo sin umbral, ambas heurísticas resultan notablemente competentes en esta instancia. Con 15h y 20h, **Greedy y Hill Climbing alcanzan ambos el óptimo exacto** (33.65 y 42.70 respectivamente): el problema es lo bastante pequeño para que incluso la estrategia voraz lo resuelva de forma óptima. La diferencia aparece únicamente en el presupuesto más amplio: con 30h el HC alcanza el óptimo (61.35) mientras el Greedy se queda en 59.30 (96.7 %). Es el único caso donde las decisiones irrevocables del Greedy le impiden la combinación óptima, y donde los *swap moves* del HC marcan la diferencia: liberan horas de recursos ya seleccionados para financiar una reorganización —la inclusión de Evaluación y validación de modelos (3h, 9.1)— que el Greedy no puede alcanzar.

El óptimo exacto reencuadra la comparación. Disponer del máximo demostrable cambia la lectura del experimento: no se trata tanto de que “el HC le gane al Greedy” como de que, *a esta escala, el problema es fácil* y ambas heurísticas son casi óptimas. El Greedy alcanza el óptimo en 2 de 3 presupuestos y el HC en los 3. La ventaja del HC es real pero marginal (un +3.5 % en el único presupuesto donde difieren) y, como se discute en la sección de limitaciones, su valor frente al Greedy es esencialmente asintótico.

Nota sobre la generalización de los resultados. El dataset de 21 recursos es suficientemente pequeño para que tanto el Greedy como el HC encuentren soluciones óptimas o near-óptimas en casi todos los casos —y para que el propio óptimo exacto se calcule en milisegundos. Con instancias más grandes (50–100 recursos y grafos de dependencias más densos), donde el cálculo exacto se vuelve intratable y el Greedy se queda corto con mayor frecuencia, la ventaja del HC sería previsiblemente más pronunciada y consistente, dado que el espacio de búsqueda crece exponencialmente mientras que el costo de los swap moves crece de forma polinomial. Los resultados aquí reportados deben interpretarse como válidos para esta instancia concreta.

Rutas óptimas — Objetivo ML Engineer

A continuación se muestran las rutas óptimas para los presupuestos de 15h y 30h, ilustrando la progresión de recursos que el sistema selecciona al disponer de más tiempo.

Cuadro 3: Ruta óptima — ML Engineer, 15h. Total: 15h | Utilidad: 33.65

#	Recurso	Horas	Utilidad (boost)
1	Álgebra lineal	5h	9.1
2	Python básico	3h	5.0
3	NumPy y Pandas	3h	10.0
4	Preprocesamiento de texto	2h	7.0
5	Visualización de datos	2h	2.5

Cuadro 4: Ruta óptima — ML Engineer, 30h. Total: 30h | Utilidad: 61.35

#	Recurso	Horas	Utilidad (boost)
1	Cálculo diferencial	4h	9.1
2	Álgebra lineal	5h	9.1
3	Python básico	3h	5.0
4	Probabilidad y estadística	4h	5.0
5	Preprocesamiento de texto	2h	7.0
6	NumPy y Pandas	3h	10.0
7	Machine learning clásico	6h	7.0
8	Evaluación y validación de modelos	3h	9.1

Las rutas respetan el ordenamiento topológico: ningún recurso aparece antes que sus prerequisites. La progresión de 15h a 30h muestra cómo el sistema incorpora gradualmente recursos de niveles más profundos del grafo de dependencias a medida que el presupuesto lo permite.

7.3 Experimento 2: Objetivo NLP Researcher

Cuadro 5: Comparación Greedy vs. Hill Climbing vs. óptimo exacto — objetivo NLP Researcher.

Presup.	Greedy			Hill Climbing				Óptimo
	Horas	Util.	% ópt	Horas	Util.	Iter.	% ópt	Util.
15h	15h	21.90	100 %	15h	21.90	8	100 %	21.90
20h	20h	27.25	100 %	20h	27.25	10	100 %	27.25
30h	30h	36.50	100 %	30h	36.50	12	100 %	36.50

El experimento NLP Researcher revela un patrón cualitativamente distinto. Los tres recursos de máxima utilidad (Transformer 9.5, Word Embeddings 8.5, RNN 8.5) requieren cadenas de dependencias de 40–52 horas —imposibles de alcanzar con presupuestos de 15h, 20h o incluso 30h.

Como consecuencia, ambos algoritmos seleccionan solo recursos fundacionales para el objetivo NLP, y **ambos alcanzan el óptimo exacto** (100 %) en los tres presupuestos: aquí no hay diferencia entre Greedy y HC. La estructura del problema —no la sofisticación del algoritmo— domina por completo el resultado.

A diferencia del objetivo ML Engineer, aquí la utilidad sí crece con el presupuesto: de 21.90 (15h) a 27.25 (20h) a 36.50 (30h). El sistema invierte cada hora adicional disponible en recursos fundacionales de utilidad modesta —incluyendo *Lógica y matemática discreta* (2.5), que un umbral de utilidad habría descartado pero que, al haber horas libres, contribuye a la utilidad total. Lo que el presupuesto extra *no* consigue es acercar los recursos objetivo de NLP, que permanecen tras cadenas de 40h o más.

7.4 Análisis Monte Carlo del Hill Climbing

La siguiente tabla presenta los resultados del análisis Monte Carlo ($N = 30$ réplicas) para el objetivo ML Engineer, que es el que mostró mayor variabilidad entre algoritmos en el experimento principal.

Cuadro 6: Análisis Monte Carlo ($N = 30$) — objetivo ML Engineer.

Presup.	Media	Desv. Est.	Mínimo	Máximo	IC 95 %
15h	33.65	0.00	33.65	33.65	[33,65, 33,65]
20h	42.70	0.00	42.70	42.70	[42,70, 42,70]
30h	61.35	0.00	61.35	61.35	[61,35, 61,35]

Los resultados son contundentes: la desviación estándar es exactamente 0.00 en los tres presupuestos, lo que significa que las 30 réplicas independientes convergen idénticamente a la misma solución. Los intervalos de confianza son degenerados $[x, x]$ porque no existe varianza entre réplicas.

Este hallazgo tiene dos interpretaciones. Primero, el espacio de soluciones para este conjunto de scores tiene un único óptimo local dominante que el HC encuentra de forma determinista independientemente del punto de inicio aleatorio: todas las semillas conducen a la misma región óptima del espacio. Segundo, esto valida con 95 % de confianza estadística que los resultados de la ejecución individual son perfectamente representativos del rendimiento esperado del algoritmo.

El óptimo exacto cierra la interpretación: ese único óptimo dominante *es* el óptimo global del problema (33.65, 42.70, 61.35), por lo que el $\text{std} = 0$ no es solo estabilidad estadística sino la consecuencia de que todas las semillas convergen al máximo demostrable. En este objetivo el Greedy también alcanza el óptimo en 15h y 20h; solo en 30h se queda en 59.30 (96.7 %), que es justo el presupuesto donde el HC conserva su ventaja.

Nota crítica sobre el $\text{std} = 0$. Un valor de desviación estándar igual a cero implica que el componente estocástico del HC —los reinicios aleatorios— no aporta variedad observable en los resultados para esta instancia. Dicho de otro modo, el algoritmo se comporta de forma efectivamente determinista en este problema concreto. Esto no invalida el análisis Monte Carlo, pero sí sugiere que los 5 reinicios aleatorios son redundantes para un dataset de 21 recursos con scores bien diferenciados: el óptimo es tan dominante que cualquier punto de inicio lo alcanza.

En instancias más grandes y con mayor densidad de restricciones, donde el paisaje de soluciones tiene múltiples óptimos locales de calidad comparable, se esperaría una desviación estándar positiva y los reinicios cobrarían valor real. El $\text{std} = 0$ es, en última instancia, otro indicador de que el tamaño del dataset limita la complejidad del problema para estos algoritmos. El único caso con varianza positiva en todo el estudio fue NLP Researcher a 15h ($\sigma = 0,09$), un primer indicio —tenue— de la estocasticidad que el HC exhibiría a mayor escala.

Rutas óptimas — Objetivo NLP Researcher

Las siguientes tablas ilustran las rutas óptimas para el objetivo NLP Researcher. El contraste con las rutas del objetivo ML Engineer es la evidencia más directa del fenómeno de *dependency drag*.

Cuadro 7: Ruta óptima — NLP Researcher, 15h. Total: 15h | Utilidad: 21.90

#	Recurso	Horas	Utilidad (boost)
1	Lógica y matemática discreta	3h	2.5
2	Cálculo diferencial	4h	5.3
3	Python básico	3h	3.4
4	NumPy y Pandas	3h	3.4
5	Preprocesamiento de texto	2h	7.3

Cuadro 8: Ruta óptima — NLP Researcher, 20h. Total: 20h | Utilidad: 27.25

#	Recurso	Horas	Utilidad (boost)
1	Lógica y matemática discreta	3h	2.5
2	Cálculo diferencial	4h	5.3
3	Álgebra lineal	5h	5.3
4	Python básico	3h	3.4
5	NumPy y Pandas	3h	3.4
6	Preprocesamiento de texto	2h	7.3

Cuadro 9: Ruta óptima — NLP Researcher, 30h. Total: 30h | Utilidad: 36.50

#	Recurso	Horas	Utilidad (boost)
1	Cálculo diferencial	4h	5.3
2	Álgebra lineal	5h	5.3
3	Python básico	3h	3.4
4	Probabilidad y estadística	4h	3.9
5	Preprocesamiento de texto	2h	7.3
6	NumPy y Pandas	3h	3.4
7	Machine learning clásico	6h	3.4
8	Evaluación y validación de modelos	3h	4.5

Nótese que ninguno de los recursos directamente relevantes para el objetivo —Arquitectura Transformer (9.5 raw), RNN/LSTM (8.5) o Word Embeddings (8.5)— aparece en ninguna de las rutas. La cadena de dependencias mínima para alcanzar Word Embeddings requiere 40h, superando incluso el presupuesto máximo evaluado. El sistema se ve obligado a llenar la ruta con recursos fundacionales de baja relevancia directa (scores de 2.5 a 5.3); el único recurso directamente relacionado con NLP que resulta alcanzable es *Preprocesamiento de texto* (7.3), por estar a un solo nivel de profundidad en el grafo de dependencias.

Esta comparación hace tangible la limitación de *dependency drag*: la misma arquitectura que produce rutas coherentes y de alta utilidad para ML Engineer genera rutas semánticamente pobres para NLP Researcher, no por un fallo del algoritmo, sino por la estructura del grafo de dependencias del dominio.

8 Limitaciones y Posibles Mejoras

8.1 Dependency drag

La limitación más significativa es el fenómeno de *dependency drag*: recursos de alta utilidad al final de cadenas largas de dependencias resultan inalcanzables con presupuestos pequeños. Para un investigador de NLP con 20h disponibles, el sistema no puede ofrecer nada directamente relevante a su objetivo —solo recursos fundacionales— porque los recursos objetivo requieren más de 40h solo en prerequisites.

Mejora propuesta: implementar un módulo de diagnóstico de alcanzabilidad que calcule el costo mínimo de cierre para los recursos de mayor utilidad e informe al usuario si ninguno es alcanzable con el presupuesto dado, sugiriendo el presupuesto mínimo necesario.

8.2 Compresión de scores en modelos pequeños

El modelo `llama-3.1-8b-instant` es propenso a comprimir scores hacia un valor central cuando recibe prompts complejos, produciendo scores similares para todos los recursos sin diferenciación útil. Este comportamiento fue reproducido experimentalmente.

Mejora propuesta: usar modelos de mayor capacidad para el scoring, o implementar comparación por pares (*pairwise ranking*) que no requiera calibración absoluta de scores.

8.3 Exploración limitada del Hill Climbing

Aunque la introducción de *swap moves* mejoró sustancialmente la exploración del HC (de 1 a 8–12 iteraciones efectivas para ML Engineer), en instancias con alto dependency drag el espacio de búsqueda efectivo sigue siendo pequeño y ambos algoritmos convergen trivialmente al óptimo.

Mejora propuesta: implementar Simulated Annealing, permitiendo aceptar movimientos que empeoran temporalmente la utilidad con probabilidad decreciente, para explorar regiones globalmente prometedoras que el HC puro rechaza.

8.4 Dataset sintético de tamaño reducido

Con 21 recursos el problema es suficientemente pequeño para que tanto el Greedy como el HC encuentren el óptimo en casi todos los casos (5 de 6 configuraciones lo alcanzan con ambos algoritmos). En datasets reales de mayor tamaño (100+ recursos) se esperaría una ventaja más pronunciada y consistente del HC.

Conviene ser explícito sobre una consecuencia de este tamaño: a la escala actual el óptimo exacto se calcula en milisegundos por branch and bound, por lo que el Hill Climbing no es estrictamente necesario aquí —el solver exacto lo domina en calidad y es viable en tiempo. La justificación del HC es *asintótica*: el knapsack con restricciones de precedencia es NP-difícil, de modo que el cálculo exacto deja de ser factible al crecer la instancia, mientras que el costo de los *swap moves* crece de forma polinomial. Los resultados de este trabajo deben leerse, por tanto, como una *validación de corrección* de las heurísticas en una instancia donde se conoce el óptimo, no como una demostración de la necesidad práctica del HC a esta escala.

Mejora propuesta: escalar el dataset a 50–100 recursos mediante generación procedural o extracción de currícula reales (roadmaps de plataformas educativas como Coursera o roadmap.sh).

9 Conclusiones

ALPS demuestra que es posible combinar técnicas de optimización combinatoria con modelos de lenguaje para construir un sistema de recomendación de rutas de aprendizaje personalizadas. El sistema resuelve formalmente un problema de satisfacción de restricciones con función objetivo, utilizando el LLM exclusivamente como componente de evaluación semántica y delegando la optimización a algoritmos clásicos.

El haber construido el óptimo exacto como referencia permite una conclusión más matizada —y más honesta— que una simple comparación entre heurísticas. A la escala de este dataset el problema resulta *fácil*: el Hill Climbing alcanza el óptimo demostrable en las seis configuraciones y el Greedy en cinco de seis. La única diferencia aparece en el presupuesto más amplio del objetivo ML Engineer (30h), donde los *swap moves* del HC le dan una ventaja de +3.5 % sobre el Greedy. La ventaja del HC es, por tanto, real pero marginal en esta instancia; su justificación frente al baseline es esencialmente asintótica y solo se manifestaría con claridad en instancias mayores donde el cálculo exacto se vuelve intratable.

Este mismo proceso de validación contra el óptimo reveló que el umbral de utilidad del diseño inicial producía soluciones subóptimas (p. ej., NLP a 20h: una ruta de 27.25 era alcanzable pero

el umbral limitaba la búsqueda a 21.40); eliminarlo corrigió el sesgo y es un ejemplo del valor de disponer de una verdad de referencia para auditar el propio sistema.

El experimento con objetivo NLP Researcher expone con claridad el fenómeno de dependency drag: la estructura del grafo de dependencias determina en gran medida qué objetivos son alcanzables dentro de un presupuesto dado, independientemente de la calidad del algoritmo.

El análisis Monte Carlo aporta rigor estadístico al componente estocástico del sistema, caracterizando la distribución de utilidades obtenidas por el HC a través de 30 réplicas independientes y proporcionando intervalos de confianza que permiten comparaciones estadísticamente válidas.

El dependency boost demuestra ser una solución efectiva al sesgo de subvaloración de recursos fundacionales, corrigiendo algorítmicamente una limitación del componente de IA sin necesidad de cambiar el modelo o el prompt.

Repositorio: [ALPS_2026](#)